Bangla-WhisperDiar: Fine-Tuning Whisper and PyAnnote for Bangla Long-Form Speech Recognition and Speaker Diarization

Mohammed Aman Bhuiyan†, Md Sazzad Hossain Adib†, Samiul Basir Bhuiyan†, Amit Chakraborty†
Department of ECE, North South University, Dhaka, Bangladesh
{mohammed.aman, sazzad.adib, samiul.bhuiyan, chakraborty.amit}@northsouth.edu
†Equal Contributions

***Abstract*—Automatic Speech Recognition (ASR) and speaker diarization in Bangla remain challenging due to long-form recordings, diverse acoustic conditions, and significant speaker variability. This work addresses these two core tasks in Bangla spoken language understanding by developing robust systems for long-form ASR and speaker diarization. For ASR (Problem 1), we fine-tune the `tugstugi_bengaliai-regional-asr_whisper-medium` model on a custom-curated dataset of approximately 15,000 chunked and aligned Bangla audio segments, employing full-weight training with extensive data augmentation including noise injection, reverb simulation, echo, clipping distortion, and pitch/time perturbation. For speaker diarization (Problem 2), we fine-tune the `pyannote/segmentation-3.0` model using PyTorch Lightning on the competition's annotated diarization dataset, swapping the fine-tuned segmentation backbone into the `pyannote/speaker-diarization-community-1` pipeline while retaining the pretrained speaker embedding and clustering components. Our ASR system achieves a Word Error Rate (WER) of 24.41%, while our diarization system achieves a Diarization Error Rate (DER) of 23.92%, both evaluated on the test set, demonstrating notable improvements over the respective pretrained baselines. We describe our complete pipeline, including data preprocessing, text normalization, audio augmentation, training strategies, inference optimization, and post-processing for both tasks. The corresponding code has been made publicly available at https://github.com/sazzadadib/BitwiseMind_DL_Sprint_4.0.**

***Index Terms*—Bangla ASR, Whisper, Speaker Diarization, PyAnnote, Fine-Tuning, Long-Form Audio, Bengali Speech Processing**

I. Introduction

Bangla (Bengali), spoken by over 230 million people worldwide, is one of the most widely spoken languages globally, yet it remains significantly under-resourced in the domain of speech technology. Long-form audio understanding, which includes both accurate transcription and speaker attribution, is critical for applications ranging from media indexing and meeting summarization to accessibility tools and legal or forensic audio analysis [1].

Despite major progress in end to end ASR and large pretrained models, most state of the art systems remain optimized for high resource languages like English, resulting in higher error rates for Bangla, especially in spontaneous, multi speaker, and diverse domain audio. Addressing this gap requires carefully curated datasets, robust modeling approaches, and evaluation strategies tailored to the unique linguistic and acoustic characteristics of Bangla. DL Sprint 4.0 poses two challenging problems on Bangla long-form audio:

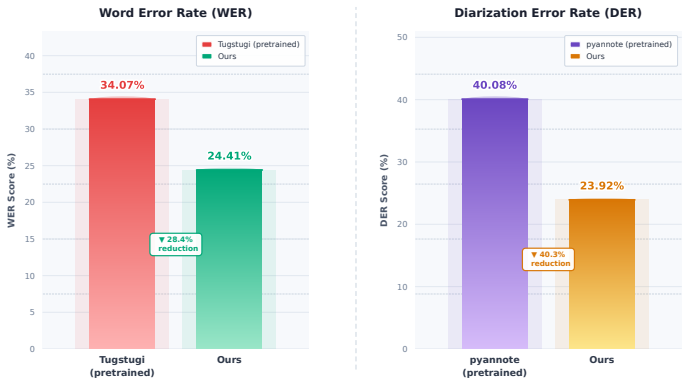

Fig. 1. Comparison of pretrained baselines and the proposed model. The proposed approach outperforms Tugstugi in WER and Pyannote in DER, indicating improved transcription and diarization performance

- **Problem 1 — Bangla Long-Form ASR:** Transcribe extended Bangla speech recordings into text, evaluated using Word Error Rate (WER).
- **Problem 2 — Speaker Diarization:** Determine "who spoke when" by producing speaker-labelled time segments, evaluated using Diarization Error Rate (DER).

Both problems present unique challenges for Bangla. The ASR task must handle dialectal variation, conversational speech, environmental noise, and the morphological complexity of the Bangla script. The diarization task must accurately segment long recordings with potentially overlapping speakers and varying acoustic conditions. In this paper, we present our approach to both problems. For ASR, we leverage OpenAI's Whisper architecture [1], specifically the medium variant pretrained on Bengali by Tugstugi [8], and perform fine-tuning with a comprehensive data augmentation pipeline. For diarization, we adopt PyAnnote.Audio's segmentation-3.0 model [3], [4] and fine-tune it on the competition dataset, integrating the fine-tuned weights into the full diarization pipeline.

Our key contributions are:

1) A complete data preprocessing pipeline for Bangla long-form audio, including automated chunking with ASR-guided ground-truth alignment using fuzzy matching.
2) A comprehensive audio augmentation strategy combining noise injection, room reverb simulation, multi-tap echo, clipping distortion, bandpass filtering, pitch shifting, and time stretching to improve model robustness. Bangla-specific text normalization including English numeral-to-Bangla word conversion and non-Bengali character removal.
3) Longform audio ASR model fine-tuning, followed by domain-adaptive fine-tuning of PyAnnote's segmentation model using checkpoint-based training and segmentation-swap inference for speaker diarization.
4) Post-processing techniques including hallucination removal for ASR and short-segment filtering for diarization.

II. Related Work

A. Automatic Speech Recognition

Modern ASR systems have shifted from traditional Hidden Markov Model (HMM) and Connectionist Temporal Classification (CTC) approaches toward end-to-end encoder-decoder architectures. OpenAI's Whisper [1] represents a significant advance in multilingual ASR, trained on 680,000 hours of weakly supervised data spanning 99 languages. Whisper uses a Transformer-based encoder-decoder architecture that converts log-Mel spectrograms to text in an autoregressive fashion. While Whisper supports Bangla, its performance on long-form conversational Bangla speech remains suboptimal without domain-specific adaptation.

BengaliAI has released several fine-tuned Whisper variants for regional Bengali ASR [2], which serve as strong starting points for further fine-tuning. Data augmentation for speech has been extensively studied, including SpecAugment [5], noise injection, and room impulse response (RIR) simulation [6], all of which we adapt for our pipeline.

B. Speaker Diarization

Speaker diarization has evolved from clustering-based methods (e.g., agglomerative hierarchical clustering on speaker embeddings) to end-to-end neural approaches. PyAnnote.Audio [3], [4] provides a modular pipeline that combines neural speaker segmentation with speaker embedding extraction and clustering. The segmentation-3.0 model uses a PyanNet architecture to produce frame-level speaker activity probabilities; these are then paired with speaker embeddings to perform full diarization.

Recent work on low-resource diarization has shown that fine-tuning pretrained segmentation models on target-domain data can significantly reduce DER [4]. Our approach follows this paradigm, adapting the multilingual pretrained model to the specific acoustic and speaker characteristics of Bangla conversational audio.

III. Methodology

A. Problem 1 — Bangla Long-Form ASR

Our ASR pipeline in Figure 2 consists of four main stages: (1) data preprocessing and chunking, (2) text normalization and cleaning, (3) model fine-tuning, and (4) inference with post-processing.

1) Data Preprocessing and Audio Chunking: Long-form audio recordings (varying from several minutes to over an hour) are first chunked into segments of at most 25 seconds, the practical input limit for Whisper's encoder. We employ a fixed-length non-overlapping chunking strategy:

1) Each training audio file is loaded at 16 kHz.
2) The audio is divided into non-overlapping 25-second segments.
3) Silent or near-silent trailing fragments (below 0.5 s) are discarded.
4) Each chunk is transcribed using the pretrained BengaliAI Whisper model to obtain an ASR hypothesis.

To obtain ground-truth labels for each chunk, we implement a **sequential fuzzy alignment** algorithm. The full ground-truth transcript is split into words, and a sequential pointer is maintained across the corpus. For each ASR-transcribed chunk, a bidirectional search window (± 5 words) and span-length variation (± 3 words) are used to find the best-matching ground-truth span via `RapidFuzz` ratio scoring. This ensures reading-order preservation while accommodating minor ASR insertions and deletions.

2) Text Normalization and Cleaning: We apply several Bangla-specific normalization steps to ensure consistency and linguistic correctness in the transcripts. First, all English-digit sequences are converted into their Bangla word equivalents using the `num2words` library, with four-digit numbers in the range 1000–2099 specifically interpreted as calendar years. Next, characters outside the Bengali Unicode block (U+0980–U+09FF) are removed, while allowing digits, whitespace, and common punctuation marks. Finally, multiple consecutive whitespace characters are collapsed into a single space to maintain clean and uniform text formatting.

3) Audio Data Augmentation: To improve robustness to real-world acoustic conditions, we apply a stochastic augmentation pipeline to approximately 30% of each training audio clip's duration. Augmentations are performed on non-overlapping random windows of 3–6 seconds. Within these windows, pink and white noise are mixed and added with a 65% probability using a randomly selected scaling factor. A multi-tap echo effect is applied with 55% probability, introducing 2–4 echo taps with random delays between 150–800,ms and decay factors ranging from 0.4 to 0.75. Room reverberation is incorporated with 60% probability through convolution with a synthetic Room Impulse Response (RIR), generated for randomly selected room sizes (small, medium, or large) and configurable RT60 decay values. Additionally, analog-to-digital clipping distortion is simulated with 30% probability. Bandpass filtering is applied with 20% probability to simulate telephone-band audio (300–3400,Hz). Pitch shifting of up

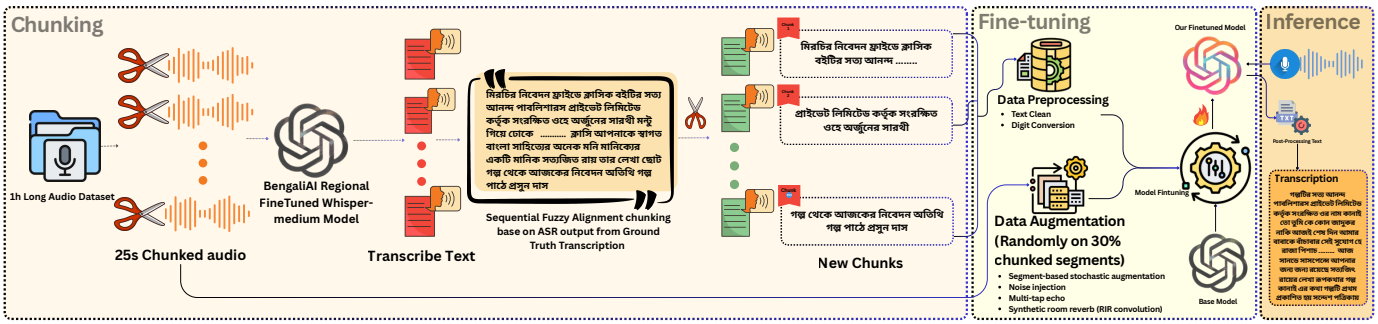

Fig. 2. End-to-end pipeline for Bangla Long-Form Speech Recognition, using fine-tuned Whisper-medium(BengaliAI) ASR model. The framework demonstrates data preprocessing, dataset chunking, model fine-tuning, and inference stages for transcribing long-form speech to standard Bangla transcription orthography.

to ± 3 semitones is performed with 25% probability using `librosa`, and time stretching is applied with 25% probability using a rate factor sampled between 0.80 and 1.20. Each augmented segment is individually peak-normalized before being reinserted into the original audio stream.

4) Model Architecture and Fine-Tuning: We use the Tugstugi [8] model as our base, which is built upon Whisper-Medium ($\sim$764M parameters) and already fine-tuned on Bengali regional speech data. To maximize adaptation capacity, we perform **full-weight fine-tuning** rather than parameter-efficient adaptation.

Training is conducted using the AdamW 8-bit optimizer adamw-bnb-8bit, which reduces VRAM usage by approximately 2.4 GB compared to standard fp32 AdamW while maintaining equivalent convergence behavior. We employ fp16 mixed precision training with a learning rate of 5×10^{-6}, scheduled using a cosine decay with 100 warmup steps. The batch size is set to 8 per GPU with 2 gradient accumulation steps, resulting in an effective batch size of 16. The model is trained for 6 epochs, with total steps computed dynamically based on dataset size. Input constraints include a maximum audio duration of 30s and a maximum transcript length of 1000 tokens. Gradient checkpointing is enabled to further reduce VRAM consumption. The best model checkpoint is selected based on the lowest Word Error Rate (WER) on the validation set. Audio inputs are converted into 80-channel log-Mel spectrograms using `WhisperFeatureExtractor`, and transcripts are tokenized with the Bangla Whisper tokenizer.

5) Inference Pipeline: During inference, test audio files are first loaded and resampled to 16,kHz mono. Each audio sample is then segmented into 25-second chunks, with zero-padding applied to the final chunk when necessary to maintain consistent input length. The segments are processed in batches of 8 using beam search decoding with num-beams=4 and max-length=448. To accelerate inference, static KV caching was enabled.

Post-processing includes NFC Unicode normalization and removal of zero-width characters to ensure clean text output. Hallucinated repetitions are mitigated through iterative regex-based deduplication targeting repeated multi-word sequences, single-word repetitions, and character-level n-gram patterns. Finally, speaker-change markers (e.g., >>) are removed from the generated transcripts.

B. Problem 2 — Speaker Diarization

Our diarization pipeline in Figure 3 consists of (1) data preparation, (2) model fine-tuning, (3) pipeline construction, and (4) post-processing.

1) Data Preparation: The competition provides annotated training files with CSV annotations containing start-time, end-time, and speaker-id fields in HH:MM:SS format. We convert these annotations into the standard formats required by PyAnnote. Specifically, we generate RTTM files to encode speaker turn segments with their corresponding start times and durations. We also create UEM files to define the full scoring region for each audio recording, spanning from time 0 to the total duration. In addition, LST files are prepared to enumerate file URIs for the train, development, and test splits, and a database.yml configuration file is constructed to enable PyAnnote's protocol system to correctly locate the associated audio, RTTM, and UEM files. For validation, the last two files in the dataset are reserved for development, while the remaining files are used for training.

2) Model Architecture: The diarization system is built upon the modular pipeline provided by PyAnnote, which comprises three primary components. First, the **Segmentation** stage uses pyannote/segmentation-3.0, a PyanNet-based neural model that generates frame-level speaker activity probabilities. It operates on 10-second audio chunks and outputs per-frame speaker presence indicators, supporting up to three simultaneous speakers. Second, the **Speaker Embedding** component employs a pretrained speaker embedding model to extract fixed-dimensional speaker representations from the detected speech segments. Finally, the **Clustering** stage applies agglomerative clustering to group segments by speaker identity across the entire recording, thereby producing the final diarization output.

3) Fine-Tuning: We fine-tune only the segmentation component of the PyAnnote pipeline, while keeping the speaker embedding and clustering components at their pretrained weights. Training is conducted using the PyTorch Lightning framework [7] on single-GPU hardware (NVIDIA Tesla T4).

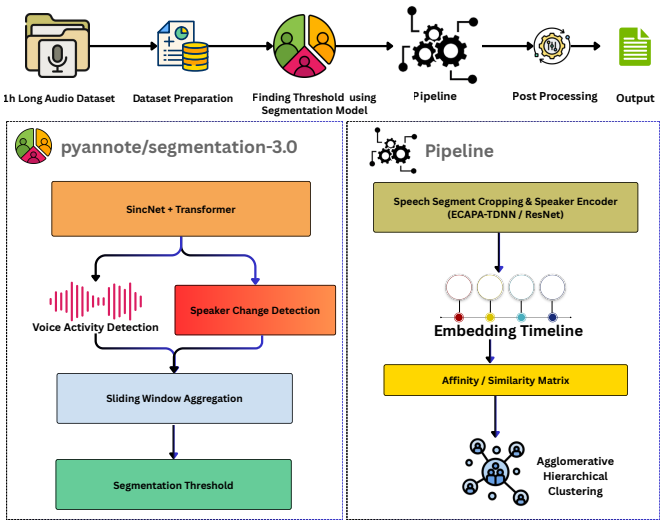

Fig. 3. Complete end-to-end pipeline for fine-tuned Bengali speaker diarization using pyannote.audio 4.0. The segmentation model (SincNet + Transformer Encoder) is fine-tuned on DL Sprint 4.0 Bengali data while the speaker embedding and clustering modules remain frozen at pretrained weights

The task is defined as `SpeakerDiarization` with 10 second audio chunks, a maximum of 3 speakers per chunk, and at most 1 speaker per frame. The model is trained for 50 epochs, with checkpointing configured to save the best model based on minimum training loss (`loss/train`), retaining both the single best checkpoint and the last checkpoint for reference.

4) Pipeline Construction: At inference time, the complete diarization pipeline is assembled by first loading the pretrained `pyannote/speaker-diarization-community-1` pipeline. The best fine-tuned segmentation checkpoint (e.g., `segmentation-epoch=39.ckpt`) is then loaded, and the segmentation `Inference` object within the pipeline is swapped with the fine-tuned model. All original pipeline parameters, including segment duration, step size, aggregation settings, and batch size, are preserved. This integration ensures that the fine-tuned segmentation model operates within the established and validated pipeline infrastructure.

5) Post-Processing: To improve output quality, we apply a minimum-duration segment filter with a 0.3 second threshold, removing spurious ultra-short segments that are likely artifacts of the segmentation model. The final diarization output is formatted as JSON, with each segment represented by start-time, end-time, and speaker-id fields.

IV. Experimental Setup

A. Dataset

1) ASR Dataset: The ASR dataset is derived from the **BengaliLoop** [9] long-form benchmark (DL Sprint 4.0). The corpus contains **191 recordings** totaling **158.6 hours** and approximately **792k words**, collected from 11 public YouTube channels, primarily Bangla drama (natok), along with news and entertainment content.

Transcripts were obtained through subtitle extraction followed by *human-in-the-loop* verification and correction to ensure alignment with spoken audio. Audio was standardized to 16 kHz mono WAV format.

For our experiments:

- Long recordings were segmented into $\leq$ 25-second chunks.
- Ground-truth transcripts were aligned to each segment.
- The final processed dataset contains approximately **15,229** samples.
- Train/validation split: **90%/10%**.

2) Diarization Dataset: The diarization dataset also is taken from the **BengaliLoop** [9] speaker diarization benchmark, consisting of **24 manually annotated recordings** totaling **22 hours** with **5,744 speaker segments**.

The official split is:

- **Train:** 10 recordings (9.5 hours, 2,612 segments).
- **Test:** 14 recordings (12.5 hours, 3,132 segments).

For our experiments:

- Training/Validation split: All files except the last 2. Last 2 files used for evaluation during training.

Annotations are provided in CSV format with start time, end time, and speaker ID. Overlapping speech is assigned to the first speaker (single-label policy), following the benchmark protocol.

B. Hardware

All experiments are conducted on Kaggle's free-tier NVIDIA Tesla T4 GPU. Key memory management strategies include:

- ASR: Gradient checkpointing, 8-bit AdamW optimizer, fp16 mixed precision.
- Diarization: PyTorch Lightning's built-in memory management with single-GPU training.

C. Evaluation Metrics

1) Word Error Rate (WER): WER measures the edit distance between predicted and reference transcriptions at the word level:

$$\text{WER} = \frac{S + D + I}{N} \tag{1}$$

where S, D, I denote the number of substitutions, deletions, and insertions, and N is the total number of words in the reference. Lower WER is better.

2) Diarization Error Rate (DER): DER is the standard metric for speaker diarization evaluation:

$$\text{DER} = \frac{\text{FA} + \text{MISS} + \text{ERROR}}{\text{TOTAL}} \tag{2}$$

where FA is false alarm (speech detected where none exists), MISS is missed speech, ERROR is speaker confusion, and TOTAL is the total reference speech duration. Lower DER is better.

3) Real-Time Factor (RTF): Inference efficiency is measured as:

$$\text{RTF} = \frac{T}{D} \tag{3}$$

where T is the total inference time and D is the total audio duration. Lower RTF indicates faster inference.

V. Results

A. Problem 1 — Bangla Long-Form ASR

Table I compares different ASR models in terms of WER. The results show that the baseline HisabTitu-BN model performs poorly (50.67%), while the pretrained Tugstugi model significantly reduces WER to 34.07%. LoRA fine-tuning on Tugstugi further improves performance to 31.32%. Our proposed fine-tuning approach achieves the best WER of 24.41%, demonstrating a substantial relative improvement over all baselines and highlighting the effectiveness of our training and adaptation strategy.

TABLE I
ASR Performance Comparison (WER %)

Model	WER (%)
1. HisabTitu-BN	50.67
2. Tugstugi (pretrained)	34.07
3. Tugstugi (LoRA fine-tune)	31.32
4. **Ours**	**24.41***

Table II quantifies the impact of post-processing on ASR performance. Without post-processing, WER is 25.76%, which is reduced to 24.41% when post-processing is applied. This confirms that post-processing contributes a measurable improvement, complementing the gains achieved through model fine-tuning.

TABLE II
Post-processing Effectiveness Analysis

Method	WER (%)
1. Without Post-Processing	25.76
2. With Post-Processing	**24.41***

B. Problem 2 — Speaker Diarization

Table III reports Diarization Error Rate (DER) across different pipelines. Traditional VAD + clustering approaches perform poorly, with DERs of 73.71% (WebRTC VAD) and 61.50% (Silero VAD). The pretrained pyannote.audio model reduces DER to 40.08%, while our fine-tuned system achieves 23.92%, showing a dramatic improvement over both classical and pretrained methods. This emphasizes the effectiveness of our adaptation and speaker modeling approach.

C. Inference Efficiency

Table IV presents the Real-Time Factor (RTF) and inference time for the proposed system on NVIDIA T4 GPUs. Using a single GPU, the long-form ASR system processes a 3-hour 38-minute recording with an RTF of **0.1659**, meaning the system runs significantly faster than real time. With the proposed inference optimization and dual-GPU parallelization with , the processing time is further reduced to **25 minutes**, achieving an RTF of **0.0190**, which demonstrates substantial acceleration. It employs **CTranslate2-optimized** Whisper models with FP16 precision, reducing GPU memory usage while increasing inference speed.

TABLE III
Speaker Diarization Performance (DER %)

Pipeline	DER (%)
WebRTC VAD + ECAPA + Clustering	73.71
Silero VAD + ECAPA + Clustering	61.50
pyannote.audio (pretrained)	40.08
Ours	**23.92***

For the speaker diarization module, the system processes the same data in 1 hour 20 minutes with an RTF of **0.1054** on a single T4 GPU. These results indicate that both the ASR and diarization components are computationally efficient and capable of real-time or faster-than-real-time processing, making the pipeline suitable for large-scale or long-duration speech analysis tasks.

TABLE IV
Inference Efficiency (Real-Time Duration)

Task	Hardware	Inference Time	RTF
Long Form ASR	T4x1 GPU	3hr 38min	0.1659
Long Form ASR (Inference Optimized)	T4x2 GPU	**25min***	0.0190
Speaker Diarization	T4x1 GPU	1hr 20min	0.1054

D. Competition Scores

Table V summarizes our final competition scores on the public and private test sets. For long-form ASR, WER is 23.58% (public) and 24.75% (private), while for speaker diarization, DER is 18.52% (public) and 26.13% (private). These results show consistent competitive performance across both datasets, confirming the robustness and generalization of our models.

TABLE V
Competition Score Summary

Task	Public (29%)	Private (71%)
Long Form ASR (WER %)	23.580	24.750
Speaker Diarization (DER %)	18.518	26.133

VI. Analysis and Discussion

A. ASR Analysis

1) Impact of Data Augmentation: The stochastic augmentation pipeline proved essential for bridging the domain gap between the pretrained model's training data (cleaner Bengali speech) and the competition's long-form conversational audio. By simulating realistic degradations (noise, reverb, echo), the model became significantly more robust to the varied acoustic conditions present in the test data. The segment-wise augmentation strategy (augmenting random 3–6 second windows rather than entire clips) was specifically designed

to preserve natural speech patterns while injecting controlled variability.

2) Text Normalization Benefits: The Bangla-specific text normalization pipeline (particularly numeral-to-word conversion) substantially improved label consistency. Without this normalization, the model would encounter the same concept represented inconsistently as both digit sequences and Bangla words, leading to ambiguous learning signals. The non-Bengali character cleaning step removed annotation artefacts that would otherwise inject noise into the training labels.

3) Chunk Alignment Quality: The sequential fuzzy-matching approach for ground-truth alignment ensures that each 25-second audio chunk receives an accurately corresponding text label, maintaining reading order while tolerating ASR errors in the alignment signal. This is critical because misaligned labels directly degrade fine-tuning quality.

4) Post-Processing Effectiveness: Whisper models are known to hallucinate, generating repetitive phrases, especially on noisy or silent audio segments. Our iterative regex-based deduplication removes three hallucination patterns: multi-word repeated phrases, single-word repetitions, and character n-gram repetitions. This post-processing step reduces WER by cleaning artefacts from the model's output without affecting valid transcriptions.

B. Diarization Analysis

1) Value of Domain-Specific Fine-Tuning: The pretrained PyAnnote segmentation model, while multilingual, was not specifically trained on Bangla conversational speech. Fine-tuning on the competition's annotated data allowed the model to learn Bangla-specific speaker turn patterns, prosodic cues, and acoustic characteristics. The improvement in DER validates the importance of domain adaptation even when starting from a strong pretrained model.

2) Segmentation-Only Fine-Tuning Strategy: Our design choice to fine-tune *only* the segmentation component and retain the pretrained speaker embedding and clustering modules proved effective. The segmentation model, which determines when" speakers are active, benefits most from domain-specific data. The speaker embedding model, which captures what" each speaker sounds like, generalizes well across languages since it operates on acoustic features rather than linguistic content.

3) Post-Processing Impact: The minimum-duration filter (0.3,s threshold) effectively removes noise-induced false speaker segments. Ultra-short segments are typically produced when the segmentation model briefly misclassifies noise or silence as speech.

C. Challenges and Limitations

- **Computational constraints:** Training on a single T4 GPU required careful memory management. Full-weight ASR training was feasible only with 8-bit optimizer and gradient checkpointing.
- **Limited diarization data:** The small number of annotated diarization files limits the fine-tuning capacity. The 2-file development split offers limited evaluation reliability.
- **Audio quality:** Some competition audio files exhibit severe noise and distortion that even augmented training cannot fully address.

VII. Conclusion

We presented a comprehensive system for Bangla long-form ASR and speaker diarization as part of DL Sprint 4.0. For ASR, our fine-tuning of Whisper-Medium with extensive domain-specific augmentation and Bangla text normalization demonstrated significant WER reduction over the pretrained baseline. For diarization, selective fine-tuning of the PyAnnote segmentation model with a pipeline-swap strategy achieved notable DER improvements while maintaining computational efficiency. Key takeaways include: (1) full-weight fine-tuning with 8-bit optimizers is viable on consumer GPUs for medium-scale Whisper models; (2) segment-wise augmentation with diverse acoustic effects improves real-world ASR robustness; (3) fine-tuning only the segmentation component of a multi-stage diarization pipeline is an effective and practical strategy; and (4) Bangla-specific text normalization is essential for reducing training-label noise.

Future work will focus on improving efficiency and robustness for real world deployment by developing joint ASR diarization architectures with pyannote.audio aligned modules to enhance speaker assignment using transcription information. We also plan to apply model compression techniques such as quantization and pruning to reduce latency and memory usage, enabling deployment on resource constrained edge devices.

References

[1] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in *Proc. ICML*, 2023.

[2] BengaliAI, "Regional Bengali ASR Whisper models," *Hugging Face Hub*, 2024. Available: https://huggingface.co/bengaliAI

[3] H. Bredin, "pyannote.audio 2.1 speaker diarization pipeline: principle, benchmark, and recipe," in *Proc. Interspeech*, 2023.

[4] A. Plaquet and H. Bredin, "Powerset multi-class cross entropy loss for neural speaker diarization," in *Proc. Interspeech*, 2023.

[5] D. S. Park, W. Chan, Y. Zhang, C.-C. Chiu, B. Zoph, E. D. Cubuk, and Q. V. Le, "SpecAugment: A simple data augmentation method for automatic speech recognition," in *Proc. Interspeech*, 2019.

[6] T. Ko, V. Peddinti, D. Povey, M. L. Seltzer, and S. Khudanpur, "A study on data augmentation of reverberant speech for robust speech recognition," in *Proc. ICASSP*, 2017.

[7] W. Falcon and The PyTorch Lightning team, "PyTorch Lightning," 2019. Available: https://github.com/Lightning-AI/lightning

[8] "bengaliAI: tugstugi-bengaliai-regional-asr-whisper-medium," Available: https://huggingface.co/bengaliAI/tugstugi_bengaliai-regional-asr_whisper-medium.

[9] "Bengali-Loop: Community Benchmarks for Long-Form Bangla ASR and Speaker Diarization," H. M. Shadman Tabib, Istiak Ahmmed Rifti, Abdullah Muhammed Amimul Ehsan, Somik Dasgupta, Md Zim Mim Siddiqee Sowdha, Abrar Jahin Sarker, Md. Rafiul Islam Nijamy, Tanvir Hossain, Mst. Metaly Khatun, Munzer Mahmood, Rakesh Debnath, Gourab Biswas, Asif Karim, Wahid Al Azad Navid, Masnoon Muztahid, Fuad Ahmed Udoy, Shahad Shahriar Rahman, Md. Tashdiqur Rahman Shifat, Most. Sonia Khatun, Mushfiqur Rahman, Md. Miraj Hasan, Anik Saha, Mohammad Ninad Mahmud Nobo, Soumik Bhattacharjee, Tusher Bhomik, Ahmmad Nur Swapnil, and Shahriar Kabir, arXiv preprint arXiv:2602.14291, 2026. Available: https://arxiv.org/abs/2602.14291.